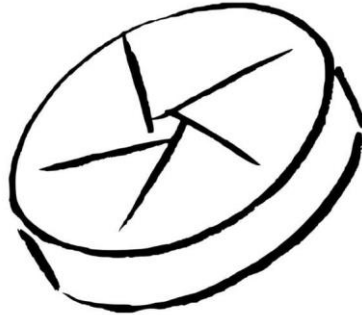


PROYECTO FIN DE CICLO



**IES RIBERA DE
LOS MOLINOS**

Departamento de Informática y Comunicaciones
Técnico Superior en Desarrollo de Aplicaciones Web

TruekApp

Autor/es: **Ramón Ángel Martínez Sánchez**

Tutor: **María Noelia Gil Valverde**

Mula, de mayo de 2026

Índice

1. Introducción.....	2
2. Objetivos del Proyecto. Definición del mismo	2
2.1 Descripción del Proyecto	2
2.2 Objetivos del proyecto	3
2.3 Alcance y limitaciones del sistema	3
2.4 Beneficios del sistema.....	4
3. Análisis del sistema	4
3.1 Obtención de información	4
3.2 Recursos necesarios	5
3.2.1 Recursos humanos.....	5
3.2.2 Recursos software y hardware	5
3.3 Planificación y Estudio de Viabilidad	6
3.3.1 Planificación del proyecto.....	6
3.3.2 Identificación de tareas	7
3.3.3 Estimación de costes	7
3.3.4 Viabilidad del proyecto	7
3.4 Casos de uso	7
♦ Caso de uso: Inicio de sesión, registro y cierre de sesión	7
Nombre: Página de inicio	8
Nombre: Iniciar sesión correctamente.....	8
Nombre: Iniciar sesión incorrecto	8
Nombre: Registrarse correctamente	9
Nombre: Registrarse incorrectamente	9
Nombre: Cerrar sesión.....	9
3.5 Casos de uso de administración	10
Nombre: Acceder al panel de administración	10
Nombre: Gestionar usuarios	10
Nombre: Gestionar servicios	11
Nombre: Cerrar sesión.....	11
3.6 Casos de uso del sistema (interacción general del usuario	11
Nombre: Ver servicios (Marketplace).....	12
Nombre: Ver detalle de un servicio	12

Nombre: Guardar servicio.....	12
Nombre: Crear servicio	13
Nombre: Editar servicio	13
Nombre: Eliminar servicio.....	13
Nombre: Solicitar trueque	14
Nombre: Gestionar solicitudes de trueque	14
Nombre: Completar trueque.....	14
Nombre: Enviar mensajes (chat).....	15
Nombre: Valorar usuario	15
Nombre: Ver perfil de usuario	15
Nombre: Ver servicios guardados.....	16
Nombre: Ver recomendaciones.....	16
4. Diseño del sistema.....	16
4.1 Diseño de los datos. Diagrama Entidad/Relación.....	16
Tabla users	17
Tabla categories	18
Tabla services.....	18
Tabla trade_requests	18
Tabla trade_messages	19
Tabla reviews	19
Tabla saved_services.....	19
Tabla notifications.....	20
Tablas auxiliares	20
4.2 Relaciones principales del sistema	20
4.3 Diseño de la interfaz	21
4.4 Diseño de procedimientos.....	27
4.5 Arquitectura general del sistema	27
5. Implementación	28
5.1 Entorno de desarrollo.....	28
Backend.....	28
Frontend	28
Herramientas adicionales	28
5.2 Arquitectura del sistema	28
5.3 Desarrollo del backend	29
Principales funcionalidades implementadas:	29
♦ Autenticación.....	29
♦ Gestión de usuarios.....	29

◆ Gestión de servicios	29
◆ Sistema de trueques	29
◆ Mensajería	29
◆ Valoraciones	30
◆ Funcionalidades adicionales	30
5.4 Desarrollo del frontend	30
Estructura principal:	30
Principales funcionalidades del frontend:	30
5.5 Integración frontend-backend	30
5.6 Control de errores y validaciones	31
Backend:.....	31
Frontend:	31
5.7 Control de versiones	31
5.8 Despliegue (pendiente / futuro)	31
6. Pruebas	32
6.1 Tipos de pruebas realizadas	32
◆ Pruebas funcionales	32
◆ Pruebas de integración.....	32
◆ Pruebas de autenticación	32
◆ Pruebas de validación de datos	33
6.2 Herramientas utilizadas	33
6.3 Ejemplos de pruebas realizadas	33
✓ Caso de prueba: Registro de usuario	33
✗ Caso de prueba: Login incorrecto.....	33
✓ Caso de prueba: Creación de servicio	33
✓ Caso de prueba: Moderación de servicio.....	34
✓ Caso de prueba: Solicitud de trueque	34
6.4 Resultados obtenidos	34
6.5 Modelo físico de la base de datos	34
6.6 Ley de protección de datos	38
6.7 Conclusión	38
7. Documentación.....	38
7.1 Documentación técnica.....	39
◆ Arquitectura del sistema	39
◆ Estructura del backend	39

♦ Estructura del frontend	39
♦ API REST	39
♦ Base de datos	40
7.2 Documentación de usuario	40
♦ Registro e inicio de sesión	40
♦ Uso de la plataforma.....	40
♦ Panel de administración.....	41
7.3 Mantenimiento del sistema	41
♦ Posibles mejoras futuras	41
7.5 Ley de protección de datos	41
7.4 Conclusión	42
8. Mantenimiento.....	42
8.1 Planificación de copias de seguridad.....	42
♦ Tipos de copias	42
♦ Frecuencia.....	42
♦ Almacenamiento	42
8.2 Escalabilidad del sistema	43
♦ Características que facilitan la escalabilidad	43
♦ Posibles mejoras de escalabilidad	43
8.3 Medidas de seguridad	43
♦ Autenticación y control de acceso	43
♦ Protección de datos.....	43
♦ Seguridad en la aplicación.....	43
8.4 Mantenimiento evolutivo.....	44
8.5 Conclusión	44
9. Mejoras futuras.....	44
9.1 Mejoras funcionales.....	44
9.2 Mejora del panel de administración.....	44
9.3 Mejora de la experiencia de usuario	45
9.4 Escalabilidad y rendimiento	45
9.5 Desarrollo de nuevas plataformas.....	45
9.6 Pruebas automatizadas.....	45
9.7 Conclusión	45
10. Conclusiones	45
10.1 Valoración del proyecto.....	46

10.2 Dificultades encontradas.....	46
10.3 Conocimientos adquiridos	46
10.4 Valoración personal	47
10.5 Anexos	47
Repositorio del proyecto.....	47
10.6 Conclusión final.....	47
11. Bibliografía.....	47
11.1 Documentación oficial.....	47
11.2 Recursos técnicos y aprendizaje	48
11.3 Herramientas utilizadas	48
11.4 Recursos adicionales.....	48

1. Introducción

El presente proyecto se desarrolla dentro del módulo profesional “**Proyecto de Desarrollo de Aplicaciones Web**” del Ciclo Formativo de Grado Superior de Desarrollo de Aplicaciones Web, impartido en el I.E.S. Ribera de los Molinos. Este módulo tiene como finalidad aplicar de forma práctica los conocimientos adquiridos a lo largo del ciclo mediante la creación de una aplicación web completa.

En este contexto, se ha desarrollado **TruekApp**, una plataforma web enfocada al intercambio de servicios entre usuarios sin necesidad de utilizar dinero. La idea parte del concepto de trueque adaptado al entorno digital, permitiendo que los usuarios puedan ofrecer habilidades o servicios propios a cambio de otros, fomentando así la colaboración y el aprovechamiento de recursos dentro de una comunidad.

La aplicación permite a los usuarios registrarse, publicar servicios tanto de oferta como de solicitud, explorar los servicios disponibles, enviar propuestas de intercambio y comunicarse mediante un sistema de mensajería interna. Una vez finalizado el intercambio, los usuarios pueden valorarse entre sí, lo que ayuda a generar confianza dentro de la plataforma. Además, se han añadido funcionalidades como el guardado de servicios, el sistema de notificaciones y un panel de administración desde el que se pueden moderar los contenidos.

A nivel técnico, la aplicación sigue una arquitectura cliente-servidor. El backend se ha desarrollado con **Laravel 11**, encargado de gestionar la lógica de negocio y exponer una API REST, mientras que el frontend se ha implementado con **React y Vite**, buscando una interfaz dinámica y fluida. La base de datos utilizada es **MySQL**, y la autenticación se gestiona mediante **Laravel Sanctum**, lo que permite trabajar con sesiones seguras basadas en tokens.

Durante el desarrollo del proyecto no solo se han aplicado los conocimientos adquiridos en el ciclo, sino que también ha sido necesario profundizar en aspectos

como la comunicación entre frontend y backend, la gestión de autenticación, la estructuración de APIs y la organización del código en aplicaciones reales.

En conjunto, TruekApp representa una aplicación funcional que cubre todo el proceso de desarrollo de software, desde el análisis inicial hasta la implementación final, reflejando la capacidad para diseñar y construir un sistema completo.

2. Objetivos del Proyecto. Definición del mismo

2.1 Descripción del Proyecto

El proyecto desarrollado, denominado **TruekApp**, consiste en una plataforma web orientada al intercambio de servicios entre usuarios sin necesidad de utilizar dinero. La idea principal es trasladar el concepto tradicional de trueque al entorno digital, permitiendo que los usuarios puedan ofrecer sus habilidades o conocimientos a cambio de otros servicios.

La aplicación está pensada para facilitar la conexión entre usuarios a través de un entorno sencillo donde pueden publicar servicios, explorar los disponibles, solicitar intercambios y comunicarse entre ellos. Además, se han implementado funcionalidades como el sistema de valoraciones, la mensajería interna, el guardado de servicios y un panel de administración para la moderación del contenido.

A nivel técnico, TruekApp sigue una arquitectura cliente-servidor. El backend se ha desarrollado con **Laravel 11**, encargado de gestionar la lógica del sistema y exponer una API REST, mientras que el frontend se ha implementado con **React y Vite**, lo que permite ofrecer una experiencia más dinámica y fluida al usuario.

2.2 Objetivos del proyecto

El objetivo principal de este proyecto es desarrollar una aplicación web funcional que permita a los usuarios intercambiar servicios de forma organizada, sencilla y segura. Para alcanzar este objetivo principal, se plantean los siguientes objetivos específicos:

- Diseñar e implementar un sistema de autenticación de usuarios seguro mediante el uso de tokens.
- Permitir a los usuarios crear, editar y eliminar servicios que ofrecen o solicitan.
- Desarrollar un sistema de búsqueda y filtrado de servicios que facilite la navegación.
- Implementar un sistema de solicitudes de intercambio (trueques) entre usuarios.
- Integrar un sistema de mensajería interna para la comunicación entre usuarios tras la aceptación de un trueque.
- Incorporar un sistema de valoraciones y reseñas que permita evaluar la calidad de los intercambios para así destacar aquellos servicios con mejor valoración y demostrar su calidad.
- Desarrollar un sistema de recomendaciones basado en el comportamiento del usuario.
- Implementar un sistema de matching que sugiera servicios compatibles entre usuarios en función de los servicios que ofrece el usuario logueado.
- Crear un panel de administración que permita la gestión y moderación de los contenidos.

- Garantizar la correcta organización y persistencia de los datos mediante una base de datos relacional.

2.3 Alcance y limitaciones del sistema

El sistema desarrollado cubre las funcionalidades esenciales necesarias para la gestión de una plataforma de intercambio de servicios, incluyendo el registro de usuarios, la publicación de servicios, la gestión de solicitudes de trueque, la comunicación entre usuarios y la valoración de los intercambios realizados.

Sin embargo, como cualquier proyecto, presenta ciertas limitaciones que deben ser consideradas:

- La aplicación está diseñada como una plataforma web, por lo que no dispone de una versión nativa para dispositivos móviles.
- El sistema de notificaciones no incluye notificaciones en tiempo real mediante tecnologías como WebSockets.
- No se ha probado el comportamiento del sistema bajo una carga elevada de usuarios simultáneos.
- La moderación de contenidos depende de la intervención manual de un administrador.
- La seguridad, aunque implementada mediante buenas prácticas, podría mejorarse con sistemas adicionales como autenticación en dos factores.

2.4 Beneficios del sistema

La implementación de TruekApp proporciona una serie de beneficios tanto a nivel funcional como social:

- Facilita el acceso a servicios sin necesidad de intercambio monetario.
- Fomenta la colaboración y la interacción entre usuarios.
- Permite el aprovechamiento de habilidades personales que no siempre tienen salida en el mercado tradicional.
- Promueve un modelo económico alternativo basado en el trueque.
- Ofrece una plataforma centralizada donde gestionar intercambios de forma organizada.
- Mejora la experiencia de usuario mediante recomendaciones personalizadas.
- Incrementa la confianza entre usuarios mediante sistemas de valoración y reseñas.

3. Análisis del sistema

El análisis del sistema tiene como objetivo identificar, definir y estructurar los requisitos necesarios para el correcto desarrollo de la aplicación TruekApp. En esta fase se busca comprender en profundidad las necesidades del sistema y los procesos que se deben implementar para garantizar su funcionamiento.

A través de este análisis se establecen los requisitos funcionales y no funcionales del sistema, que permite desarrollar una solución coherente, organizada y de fácil mantenimiento a lo largo del tiempo y para cualquier usuario que acceda al código. Además, se identifican los elementos clave que intervienen en el sistema, tales como usuarios, servicios, solicitudes de intercambio y comunicaciones entre usuarios.

El análisis previo permite reducir errores en fases posteriores del desarrollo, optimizar la estructura del sistema y asegurar que la aplicación cumpla con los objetivos planteados.

3.1 Obtención de información

Para analizar el análisis del sistema, se han utilizado técnicas de recopilación de información para comprender las necesidades del proyecto y definir sus funcionalidades.

Entre las técnicas utilizadas destacan:

- **Observación directa:** análisis de plataformas similares que permiten la interacción entre usuarios, como aplicaciones de compraventa o intercambio de servicios, para identificar funcionalidades clave y buenas prácticas.
- **Investigación online:** consulta de documentación técnica, foros de desarrollo y ejemplos reales de aplicaciones web con funcionalidades similares, lo que ha permitido definir la arquitectura y las tecnologías más adecuadas.
- **Simulación de uso:** planteamiento de distintos escenarios de uso de la aplicación, como la creación de servicios, solicitud de intercambios o comunicación entre usuarios, para detectar necesidades y posibles mejoras.
- **Experiencia personal:** conocimiento adquirido durante el ciclo formativo, especialmente en el desarrollo de aplicaciones web, que ha servido como base para la toma de decisiones técnicas.

Estas técnicas han permitido definir de forma clara los requisitos del sistema y orientar el desarrollo hacia una solución realista y funcional.

3.2 Recursos necesarios

3.2.1 Recursos humanos

El desarrollo del proyecto ha sido realizado por una única persona, el autor de este, encargándose de todas las fases del desarrollo, incluyendo:

- Análisis y diseño del sistema
- Desarrollo del backend
- Desarrollo del frontend
- Diseño de la base de datos
- Pruebas y validación del sistema
- Documentación del proyecto

Aunque el proyecto ha sido desarrollado de forma individual, se ha contado con el apoyo puntual del tutor para la orientación y resolución de dudas durante el desarrollo.

3.2.2 Recursos software y hardware

Para el desarrollo del proyecto se han utilizado los siguientes recursos:

Software:

- Sistema operativo Windows

- Entorno de desarrollo Visual Studio Code
- Backend: Laravel 11
- Frontend: React + Vite
- Gestor de base de datos MySQL (phpMyAdmin)
- Control de versiones: Git y GitHub
- Cliente HTTP para pruebas de API (Bruno/Postman)
- Navegadores web (Brave)

Hardware:

- Ordenador personal con capacidad suficiente para ejecutar entornos de desarrollo web
- Conexión a Internet para el uso de herramientas online y repositorios

3.3 Planificación y Estudio de Viabilidad

3.3.1 Planificación del proyecto

El desarrollo del proyecto se ha organizado en distintas fases, siguiendo un enfoque estructurado:

1. Planteamiento del proyecto
2. Planificación y creación de información del proyecto
3. Diseño y modelado de la base de datos
4. Implementación del backend
5. Desarrollo del frontend
6. Implementación de funcionalidades principales
7. Panel de administración
8. Pruebas y depuración
9. Despliegue y ajustes finales
10. Documentación

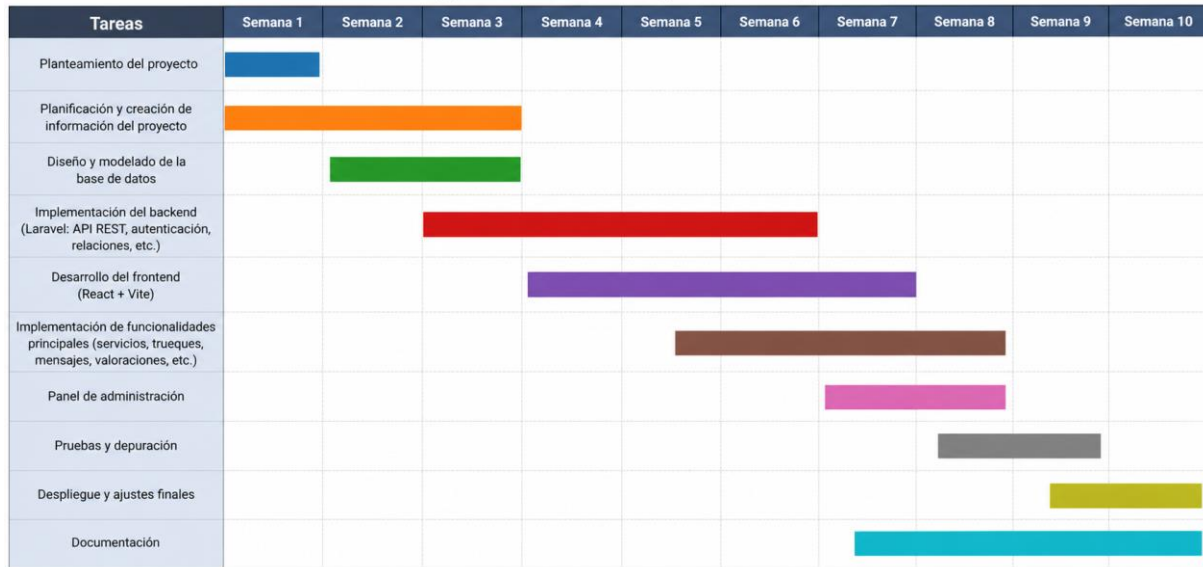
Para la organización temporal del proyecto se ha seguido una planificación progresiva, adaptándose al calendario del curso y a la carga de trabajo disponible.

Aunque no se ha utilizado una herramienta formal de gestión de proyectos, se ha realizado una planificación basada en objetivos semanales, lo que ha permitido avanzar de forma constante.

Diagrama de Gantt del desarrollo del proyecto TruekApp.

En él se representan las distintas fases del proyecto, desde el planteamiento inicial hasta la documentación final.

Diagrama de Gantt - Proyecto TruekApp



3.3.2 Identificación de tareas

Las tareas del proyecto se han definido de forma más concreta a partir de las fases generales, centrándose en los trabajos técnicos necesarios para el desarrollo del sistema:

- Diseño y estructuración de la base de datos (tablas, relaciones y restricciones)
- Implementación del sistema de autenticación y gestión de usuarios
- Desarrollo de la API REST en Laravel
- Creación de endpoints para servicios, trueques, valoraciones y mensajes
- Desarrollo de la interfaz de usuario con React
- Integración entre frontend y backend mediante peticiones HTTP
- Implementación del sistema de mensajería entre usuarios
- Desarrollo del panel de administración para moderación de contenido
- Pruebas funcionales del sistema y corrección de errores
- Preparación del proyecto para su despliegue

Esta organización ha permitido abordar el desarrollo de forma progresiva, separando claramente las distintas responsabilidades dentro del proyecto.

3.3.3 Estimación de costes

Al tratarse de un proyecto académico, no se han generado costes económicos directos, ya que se han utilizado herramientas gratuitas o de uso libre.

Sin embargo, se puede considerar como coste principal el tiempo dedicado al desarrollo, incluyendo el aprendizaje, la implementación y la resolución de problemas técnicos.

3.3.4 Viabilidad del proyecto

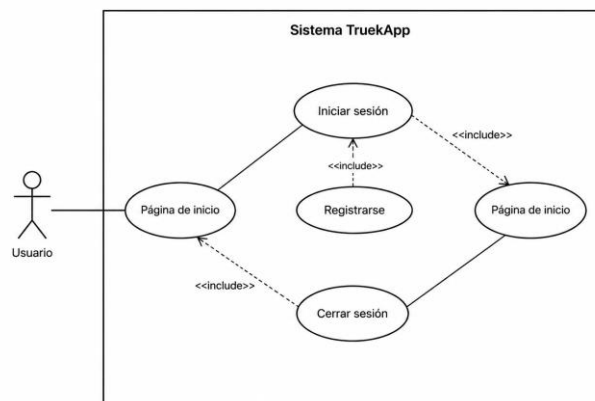
El proyecto TruekApp es viable desde diferentes puntos de vista:

- **Viabilidad técnica:** se han utilizado tecnologías actuales y ampliamente documentadas, lo que facilita su desarrollo y mantenimiento.
- **Viabilidad económica:** el desarrollo no requiere inversión económica significativa.
- **Viabilidad operativa:** el sistema es funcional y puede ser utilizado por usuarios sin conocimientos técnicos.
- **Viabilidad de crecimiento:** el sistema puede ampliarse en el futuro con nuevas funcionalidades.

3.4 Casos de uso

A continuación se describen los principales casos de uso del sistema TruekApp.

◊ Caso de uso: Inicio de sesión, registro y cierre de sesión



Nombre: *Página de inicio*

Actores: Usuario

Descripción:

El usuario accede a la plataforma sin estar autenticado.

Precondiciones:

No se requiere que el usuario esté registrado.

Curso normal del caso de uso:

- El usuario entra en la página principal.

Postcondiciones:

El usuario puede navegar o iniciar sesión.

Nombre: *Iniciar sesión correctamente*

Actores: Usuario

Descripción:

El usuario inicia sesión en la plataforma utilizando sus credenciales.

Precondiciones:

El usuario debe estar registrado en el sistema.

Curso normal del caso de uso:

- El usuario accede al formulario de login.
- Introduce su correo y contraseña.
- Pulsa el botón de iniciar sesión.

Postcondiciones:

El usuario accede al sistema.

Nombre: Iniciar sesión incorrecto

Actores: Usuario

Descripción:

El usuario introduce credenciales incorrectas.

Precondiciones:

El usuario debe estar registrado.

Curso normal del caso de uso:

- Introduce datos erróneos.
- El sistema valida y detecta error.

Postcondiciones:

El sistema muestra mensaje de error.

Nombre: Registrarse correctamente

Actores: Usuario

Descripción:

El usuario crea una nueva cuenta.

Precondiciones:

El correo no debe estar registrado.

Curso normal del caso de uso:

- Accede al formulario de registro.
- Introduce datos requeridos.
- Envía el formulario.

Postcondiciones:

Cuenta creada correctamente.

Nombre: Registrarse incorrectamente

Actores: Usuario

Descripción:

El usuario intenta registrarse con datos inválidos.

Precondiciones:

El correo ya existe o datos incorrectos.

Curso normal del caso de uso:

- Introduce datos erróneos.
- El sistema detecta error.

Postcondiciones:

Se muestra mensaje de error.

Nombre: *Cerrar sesión*

Actores: Usuario

Descripción:

El usuario cierra su sesión activa.

Precondiciones:

Usuario autenticado.

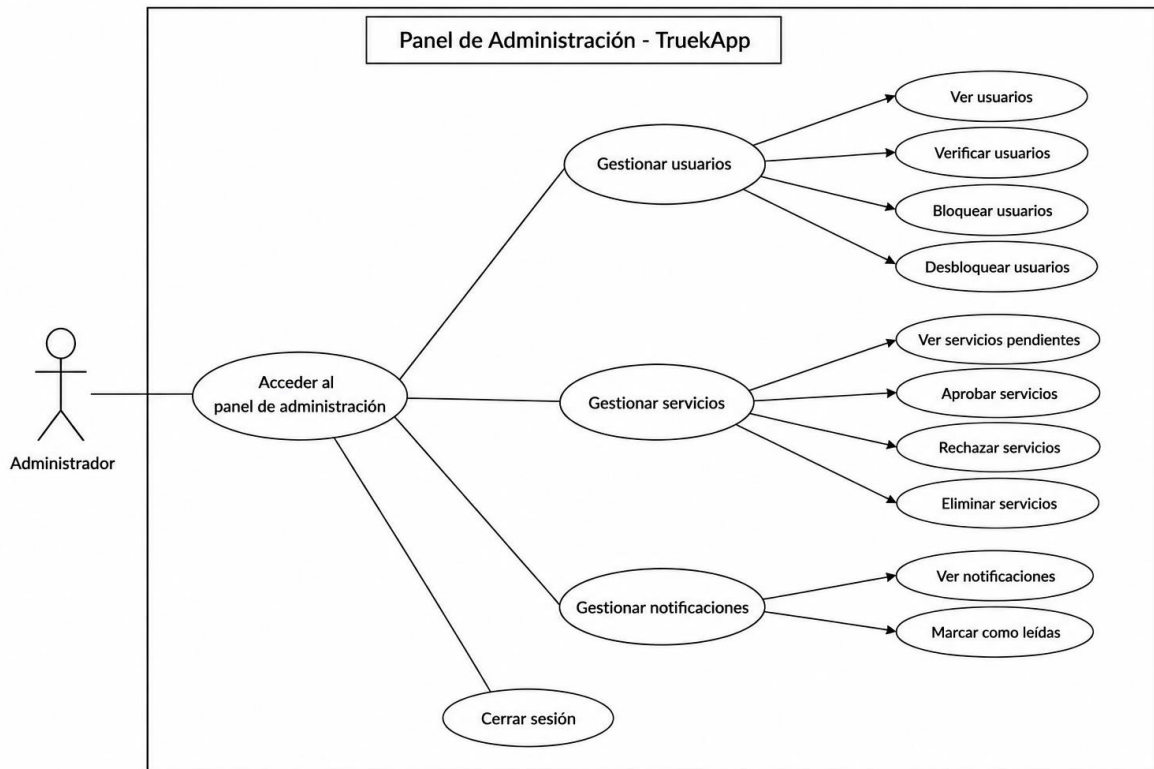
Curso normal del caso de uso:

- Pulsa botón cerrar sesión.

Postcondiciones:

El usuario vuelve a la pantalla inicial.

3.5 Casos de uso de administración



Nombre: *Acceder al panel de administración*

Actores: Administrador

Descripción:

El administrador accede al panel de control.

Precondiciones:

Debe tener rol ADMIN.

Curso normal del caso de uso:

- Inicia sesión
- Accede al panel automáticamente

Postcondiciones:

Panel cargado correctamente.

Nombre: *Gestionar usuarios*

Actores: Administrador

Descripción:

Permite administrar usuarios del sistema.

Precondiciones:

Acceso al panel admin.

Curso normal del caso de uso:

- Ver usuarios
- Verificar usuario
- Bloquear usuario

Postcondiciones:

Usuarios actualizados.

Nombre: *Gestionar servicios*

Actores: Administrador

Descripción:

Permite moderar servicios.

Precondiciones:

Existencia de servicios.

Curso normal del caso de uso:

- Ver servicios
- Aprobar / rechazar
- Eliminar

Postcondiciones:

Estado actualizado.

Nombre: *Cerrar sesión*

Actores: Administrador

Descripción:

Finaliza sesión del administrador.

Precondiciones:

Sesión activa.

Curso normal del caso de uso:

- Pulsar cerrar sesión

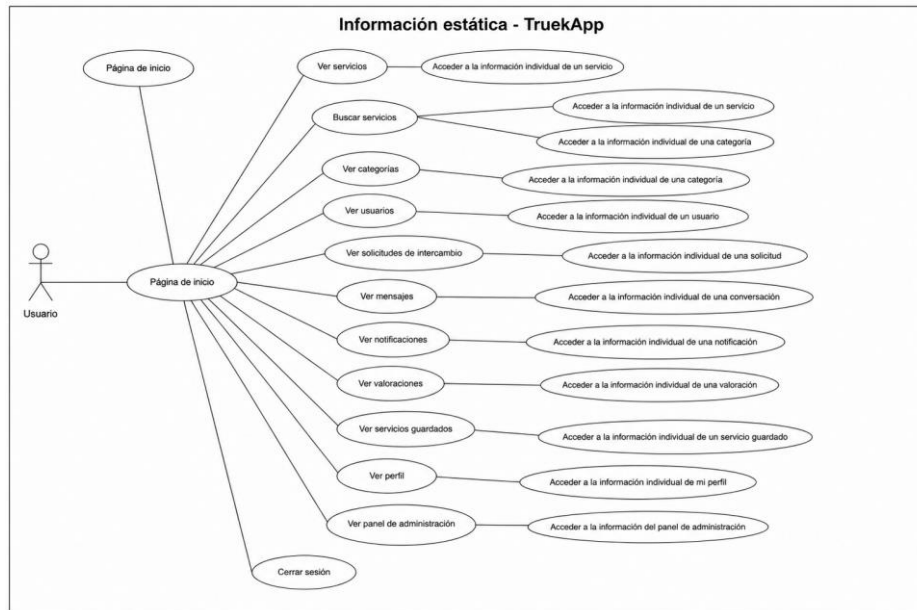
Postcondiciones:

Salida del sistema.

3.6 Casos de uso del sistema (interacción general del usuario)

Este apartado describe las acciones principales que puede realizar un usuario dentro de la plataforma TruekApp, desde la navegación básica hasta la interacción con servicios, trueques y perfiles.

TruekApp



Nombre: Ver servicios (Marketplace)

Actores: Usuario

Descripción:

El usuario accede al listado de servicios publicados en la plataforma.

Precondiciones:

No es necesario estar autenticado.

Curso normal del caso de uso:

- El usuario entra en la página principal.
- Visualiza los servicios disponibles.
- Puede usar filtros (categoría, tipo, ubicación).

Postcondiciones:

El usuario obtiene una lista de servicios disponibles.

Nombre: Ver detalle de un servicio

Actores: Usuario

Descripción:

El usuario consulta la información detallada de un servicio.

Precondiciones:

El servicio debe estar aprobado y activo.

Curso normal del caso de uso:

- El usuario selecciona un servicio.
- Accede a la página de detalle.

Postcondiciones:

El usuario visualiza toda la información del servicio.

Nombre: Guardar servicio

Actores: Usuario

Descripción:

El usuario guarda un servicio en su lista personal.

Precondiciones:

Usuario autenticado.

Curso normal del caso de uso:

- El usuario pulsa "Guardar servicio".

Postcondiciones:

El servicio queda almacenado en favoritos.

Nombre: Crear servicio

Actores: Usuario

Descripción:

El usuario crea un nuevo servicio para ofrecer o solicitar.

Precondiciones:

Usuario autenticado.

Curso normal del caso de uso:

- Accede a "Crear servicio".
- Rellena el formulario.
- Envía el servicio.

Postcondiciones:

El servicio queda en estado PENDING.

Nombre: Editar servicio

Actores: Usuario

Descripción:

El usuario modifica un servicio propio.

Precondiciones:

Ser propietario del servicio.

Curso normal del caso de uso:

- Accede a editar servicio.
- Modifica datos.
- Guarda cambios.

Postcondiciones:

El servicio se actualiza (vuelve a PENDING de validación por el admin).

Nombre: Eliminar servicio

Actores: Usuario

Descripción:

El usuario elimina uno de sus servicios.

Precondiciones:

Ser propietario del servicio.

Curso normal del caso de uso:

- Accede a sus servicios.
- Elimina uno.

Postcondiciones:

El servicio desaparece del sistema.

Nombre: Solicitar trueque

Actores: Usuario

Descripción:

El usuario solicita un intercambio con otro usuario.

Precondiciones:

- Usuario autenticado
- Servicio propio activo
- Servicio objetivo aprobado

Curso normal del caso de uso:

- Selecciona un servicio.
- Elige uno propio como oferta.
- Envía solicitud.

Postcondiciones:

Se crea un TradeRequest en estado PENDING.

Nombre: Gestionar solicitudes de trueque

Actores: Usuario

Descripción:

El usuario gestiona las solicitudes recibidas o enviadas.

Precondiciones:

Existencia de solicitudes.

Curso normal del caso de uso:

- Visualiza solicitudes.
- Acepta o rechaza si es receptor.
- Cancela si es emisor.

Postcondiciones:

El estado del trueque cambia, si acepta se abre un chat entre el solicitante y el receptor.

Nombre: Completar trueque

Actores: Usuario

Descripción:

El usuario marca un intercambio como completado.

Precondiciones:

Trueque en estado ACCEPTED.

Curso normal del caso de uso:

- Accede al trueque.
- Marca como completado.

Postcondiciones:

El trueque pasa a COMPLETED y se abre un sistema de reseña con puntuación.

Nombre: Enviar mensajes (chat)

Actores: Usuario

Descripción:

Los usuarios se comunican tras aceptar un trueque.

Precondiciones:

Trueque ACCEPTED.

Curso normal del caso de uso:

- Accede al chat.
- Envía mensajes.

Postcondiciones:

Los mensajes quedan registrados.

Nombre: Valorar usuario

Actores: Usuario

Descripción:

El usuario valora a otro tras completar un trueque.

Precondiciones:

Trueque COMPLETED.

Curso normal del caso de uso:

- Accede a valoración.
- Introduce puntuación y comentario.

Postcondiciones:

Se crea una review.

Nombre: Ver perfil de usuario

Actores: Usuario

Descripción:

El usuario consulta el perfil de otro usuario a través de un servicio.

Precondiciones:

El usuario debe existir y tener al menos un servicio publicado.

Curso normal del caso de uso:

- Accede al perfil.
- Visualiza servicios y valoraciones.

Postcondiciones:

Información mostrada correctamente.

Nombre: Ver servicios guardados

Actores: Usuario

Descripción:

El usuario consulta sus servicios guardados.

Precondiciones:

Usuario autenticado.

Curso normal del caso de uso:

- Accede a la sección de guardados.

Postcondiciones:

Lista de servicios guardados mostrada.

Nombre: Ver recomendaciones

Actores: Usuario

Descripción:

El usuario ve en la home recomendaciones personalizadas.

Precondiciones:

Usuario autenticado.

Curso normal del caso de uso:

- Accede a la home.
- Visualiza recomendaciones.

Postcondiciones:

Se muestran servicios relevantes.

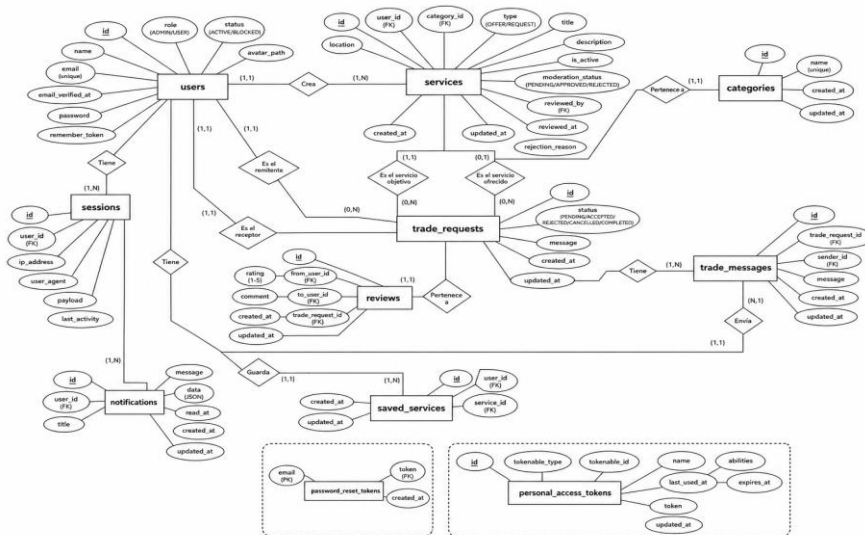
4. Diseño del sistema

El diseño del sistema define la estructura interna de TruekApp, estableciendo cómo se organizan los datos, cómo se comunican los distintos componentes de la aplicación y cómo se distribuyen las responsabilidades entre frontend, backend y base de datos. TruekApp sigue una arquitectura cliente-servidor, separando claramente la interfaz de usuario, desarrollada en React, de la lógica de negocio, desarrollada en Laravel mediante una API REST.

4.1 Diseño de los datos. Diagrama Entidad/Relación

Para el diseño de los datos se ha utilizado un modelo relacional implementado en MySQL. La base de datos está formada por distintas tablas que representan las entidades principales del sistema, así como tablas auxiliares necesarias para la autenticación, sesiones y recuperación de contraseña.

A continuación, se muestra el diagrama Entidad/Relación de la base de datos del proyecto:



Las tablas principales del sistema son las siguientes:

Tabla users

La tabla users almacena la información de los usuarios registrados en la plataforma. Campos principales:

- id: identificador único del usuario.
- name: nombre del usuario.
- email: correo electrónico único.
- email_verified_at: fecha de verificación del correo electrónico.
- password: contraseña cifrada.
- remember_token: token usado por Laravel para recordar sesión.
- role: rol del usuario, pudiendo ser ADMIN o USER.
- status: estado del usuario.
- avatar_path: ruta del avatar del usuario.
- created_at y updated_at: fechas de creación y actualización.

Esta tabla se relaciona con servicios, solicitudes de trueque, valoraciones, mensajes, notificaciones y servicios guardados.

Tabla categories

La tabla `categories` almacena las categorías disponibles para clasificar los servicios publicados.

Campos principales:

- `id`: identificador único de la categoría.
- `name`: nombre único de la categoría.
- `created_at` y `updated_at`: fechas de creación y actualización.

Una categoría puede estar asociada a muchos servicios.

Tabla `services`

La tabla `services` almacena los servicios publicados por los usuarios, tanto servicios ofrecidos como servicios solicitados.

Campos principales:

- `id`: identificador único del servicio.
- `user_id`: usuario propietario del servicio.
- `category_id`: categoría del servicio.
- `type`: tipo de servicio, pudiendo ser `OFFER` o `REQUEST`.
- `title`: título del servicio.
- `description`: descripción del servicio.
- `location`: ubicación asociada.
- `is_active`: indica si el servicio está visible o no.
- `moderation_status`: estado de moderación, pudiendo ser `PENDING`, `APPROVED` o `REJECTED`.
- `reviewed_by`: administrador que revisó el servicio.
- `reviewed_at`: fecha de revisión.
- `rejection_reason`: motivo de rechazo.
- `created_at` y `updated_at`: fechas de creación y actualización.

Esta tabla es una de las más importantes del sistema, ya que permite construir el marketplace de servicios. Además, incluye campos de moderación para que un administrador pueda aprobar o rechazar servicios antes de que sean visibles públicamente.

Tabla `trade_requests`

La tabla `trade_requests` representa las solicitudes de trueque entre usuarios.

Campos principales:

- `id`: identificador único de la solicitud.
- `requester_id`: usuario que realiza la solicitud.
- `target_service_id`: servicio que el usuario quiere recibir.
- `offer_service_id`: servicio que el usuario ofrece a cambio.
- `status`: estado del trueque, pudiendo ser `PENDING`, `ACCEPTED`, `REJECTED`, `CANCELLED` o `COMPLETED`.
- `message`: mensaje inicial opcional.
- `created_at` y `updated_at`: fechas de creación y actualización.

Esta tabla permite controlar el ciclo de vida de una solicitud de intercambio. Además, se aplica una restricción única sobre `requester_id` y `target_service_id` para evitar solicitudes duplicadas al mismo servicio.

Aunque `offer_service_id` permite valor nulo a nivel de base de datos, en la lógica actual del sistema se valida como obligatorio para poder solicitar un trueque.

Tabla `trade_messages`

La tabla `trade_messages` almacena los mensajes del chat asociado a una solicitud de trueque.

Campos principales:

- `id`: identificador único del mensaje.
- `trade_request_id`: solicitud de trueque a la que pertenece el mensaje.
- `sender_id`: usuario que envía el mensaje.
- `message`: contenido del mensaje.
- `created_at` y `updated_at`: fechas de creación y actualización.

Esta tabla permite que los usuarios puedan comunicarse cuando una solicitud de trueque ha sido aceptada.

Tabla `reviews`

La tabla `reviews` almacena las valoraciones realizadas entre usuarios tras completar un trueque.

Campos principales:

- `id`: identificador único de la valoración.
- `from_user_id`: usuario que realiza la valoración.
- `to_user_id`: usuario que recibe la valoración.
- `trade_request_id`: trueque asociado a la valoración.
- `rating`: puntuación de 1 a 5.
- `comment`: comentario opcional.
- `created_at` y `updated_at`: fechas de creación y actualización.

La tabla incluye una restricción única sobre `from_user_id` y `trade_request_id`, evitando que un usuario valore más de una vez el mismo trueque.

Tabla `saved_services`

La tabla `saved_services` permite que los usuarios guarden servicios como favoritos.

Campos principales:

- `id`: identificador único.
- `user_id`: usuario que guarda el servicio.
- `service_id`: servicio guardado.
- `created_at` y `updated_at`: fechas de creación y actualización.

Esta tabla incluye una restricción única sobre `user_id` y `service_id`, evitando que un usuario guarde el mismo servicio más de una vez.

Tabla notifications

La tabla notifications almacena notificaciones internas del sistema para los usuarios.

Campos principales:

- id: identificador único de la notificación.
- user_id: usuario receptor de la notificación.
- type: tipo de notificación.
- title: título de la notificación.
- message: mensaje de la notificación.
- read_at: fecha de lectura.
- created_at y updated_at: fechas de creación y actualización.

Tablas auxiliares

Además de las tablas principales del dominio de la aplicación, existen tablas auxiliares generadas o utilizadas por Laravel y Sanctum:

- personal_access_tokens: almacena los tokens de autenticación usados por Laravel Sanctum.
- password_reset_tokens: gestiona tokens para recuperación de contraseña.
- sessions: almacena información de sesiones.

Estas tablas no forman parte directa de la lógica principal de trueques, pero son necesarias para la autenticación, seguridad y funcionamiento interno del sistema.

4.2 Relaciones principales del sistema

Las relaciones más importantes entre las tablas son:

- Un usuario puede crear muchos servicios.
- Un servicio pertenece a un usuario.
- Un servicio puede pertenecer a una categoría.
- Un usuario puede realizar muchas solicitudes de trueque.
- Una solicitud de trueque tiene un servicio objetivo y un servicio ofrecido.
- Una solicitud de trueque puede tener muchos mensajes.
- Un mensaje pertenece a un usuario y a una solicitud de trueque.
- Un usuario puede valorar a otro usuario tras completar un trueque.
- Un usuario puede guardar muchos servicios.
- Un usuario puede recibir muchas notificaciones.

Este diseño permite representar correctamente las funcionalidades principales de TruekApp, manteniendo la integridad de los datos y evitando duplicidades mediante claves foráneas y restricciones únicas.

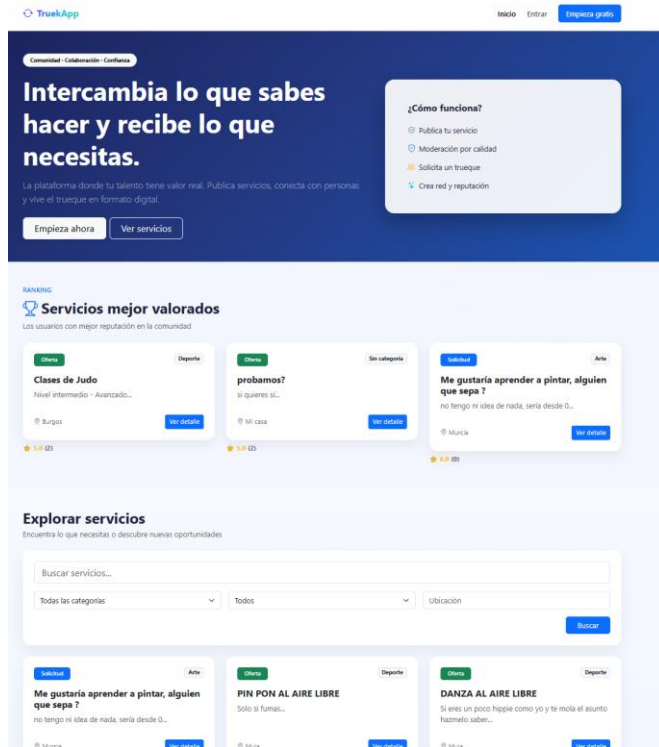
4.3 Diseño de la interfaz

TruekApp

La interfaz de TruekApp ha sido diseñada con el objetivo de ofrecer una experiencia clara, moderna y fácil de utilizar. Para ello se ha utilizado React junto con Bootstrap, permitiendo construir una interfaz responsive y organizada en componentes reutilizables.

Las pantallas principales de la aplicación son:

- Página de inicio con marketplace de servicios.



- Login y registro de usuarios.

TruekApp



[Inicio](#) [Entrar](#) [Empieza gratis](#)

Bienvenido de nuevo

Conecta con tu comunidad de intercambio.

Email

Password

Iniciar sesión

¿No tienes cuenta? [Regístrate](#)

TruekApp
Intercambia talento, crea comunidad y transforma tiempo en valor compartido.
[Publicar servicio](#)

Navegación
[Inicio](#)
[Servicios](#)
[Dashboard](#)
[Mis servicios](#)
[Administración](#)

Contacto
E2: susant@truekapp.local
[📧](#) [📱](#) [🌐](#)

© 2028 TruekApp - TFG Plataforma de Trueque de Servicios

[Privacidad](#) [Términos](#) [Soporte](#)



[Inicio](#) [Entrar](#) [Empieza gratis](#)

Crea tu cuenta

Empieza a intercambiar servicios con tu comunidad.

Nombre

Email

Password

Regístrate

¿Ya tienes cuenta? [Inicia sesión](#)

TruekApp
Intercambia talento, crea comunidad y transforma tiempo en valor compartido.
[Publicar servicio](#)

Navegación
[Inicio](#)
[Servicios](#)
[Dashboard](#)
[Mis servicios](#)
[Administración](#)

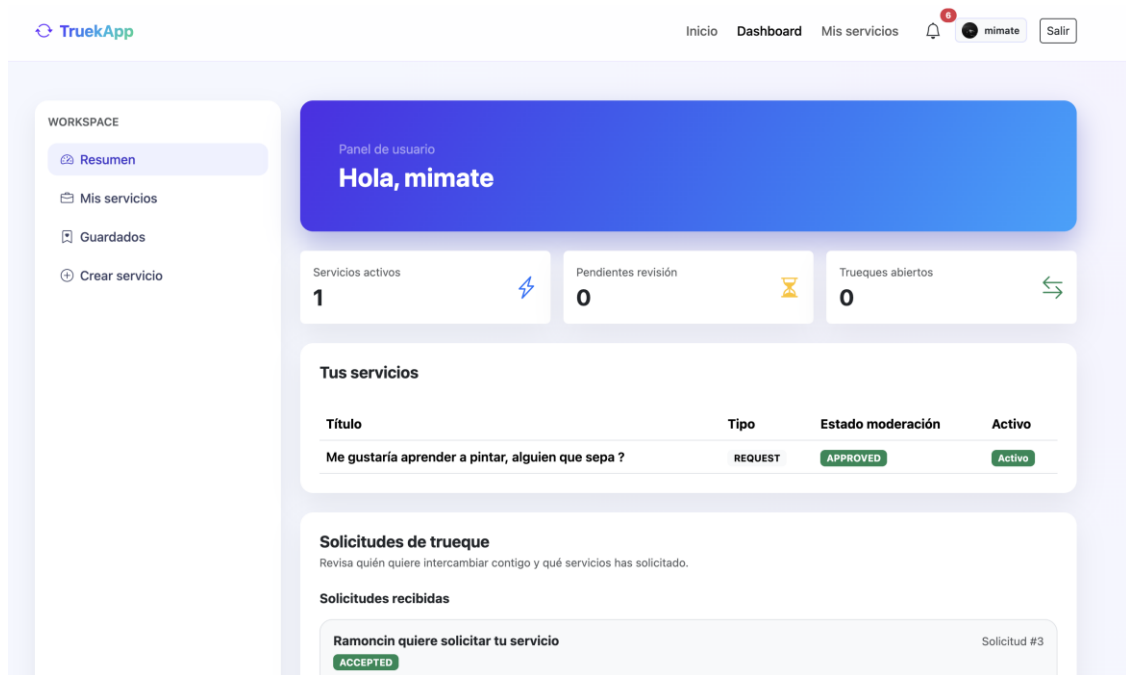
Contacto
E2: susant@truekapp.local
[📧](#) [📱](#) [🌐](#)

© 2028 TruekApp - TFG Plataforma de Trueque de Servicios

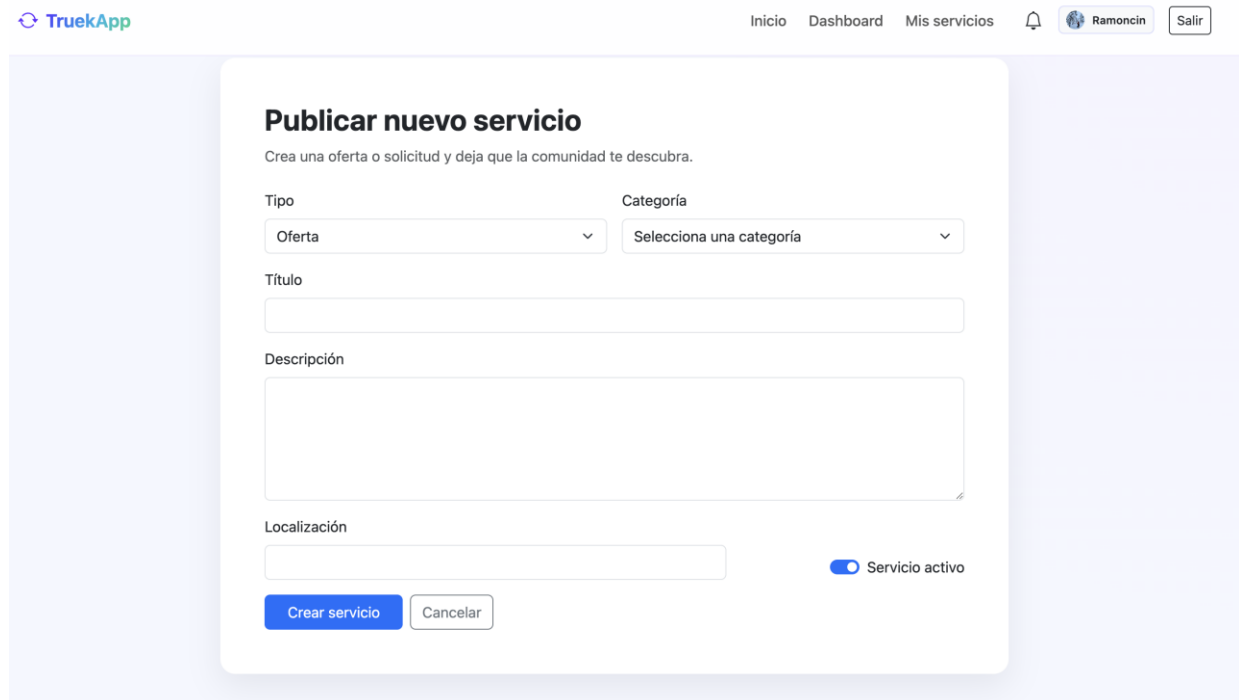
[Privacidad](#) [Términos](#) [Soporte](#)

- Dashboard del usuario.

TruekApp



- Página de creación de servicios.



- Página de edición de servicios.

TruekApp

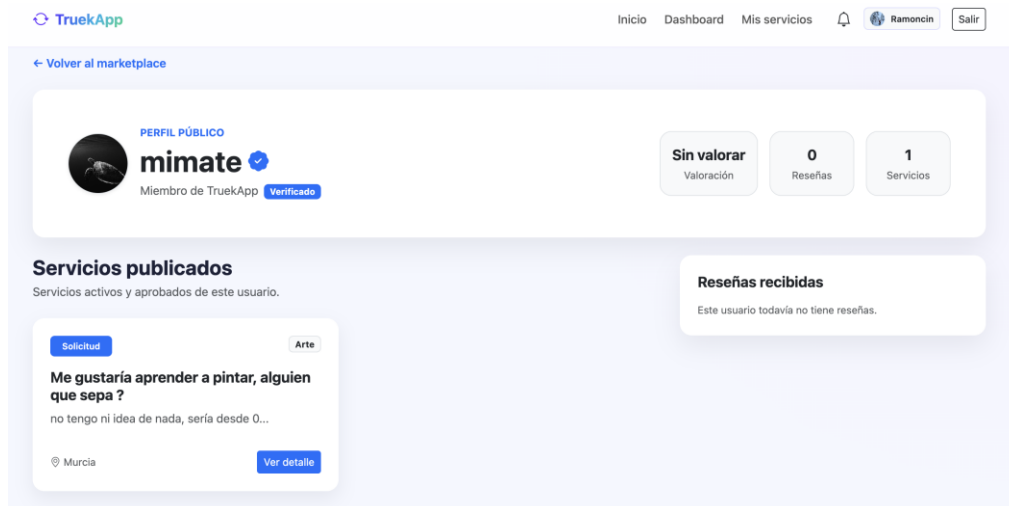
The screenshot shows the 'Editar servicio' (Edit service) form in the TruekApp interface. The form is titled 'Editar servicio' and includes a sub-header 'Modifica la información, cambia su visibilidad o desactívalo de la vista pública.' Below this, there is a light blue box with instructions: 'Si cambias título, descripción, tipo, categoría o localización, el servicio volverá a quedar pendiente de revisión. Si solo cambias la visibilidad, no necesitará nueva moderación.' The form fields include: 'Tipo' (Offer) and 'Categoría' (Sports) dropdown menus; 'Título' (Title) with the text 'Clases de tenis'; 'Descripción' (Description) with the text 'Nivel básico, para los más pequeños'; 'Localización' (Location) with the text 'Zaragoza'; and a 'Servicio activo' (Service active) toggle switch. At the bottom, there are 'Guardar cambios' (Save changes) and 'Cancelar' (Cancel) buttons.

- Página de detalle de servicio.

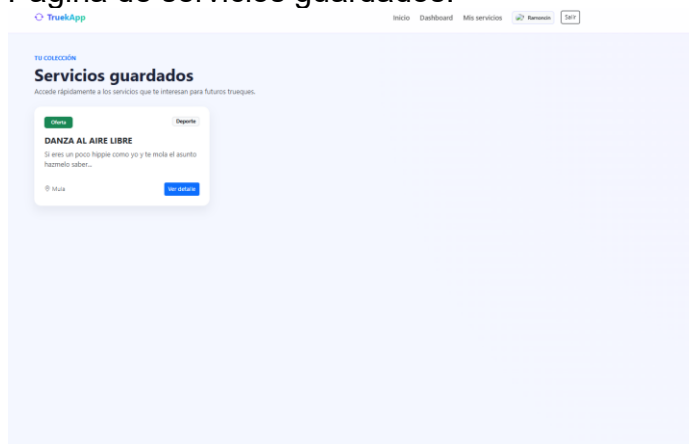
The screenshot shows the service detail page in the TruekApp interface. The page is titled 'Me gustaría aprender a pintar, alguien que sepa ?' and includes a sub-header 'no tengo ni idea de nada, sería desde 0'. The service is published by 'mimate' (Member of TruekApp) and is located in 'Murcia'. The service is categorized as 'Arte' (Art) and is a 'Solicitud' (Request). The page also shows the 'Publicado por' (Published by) section with the user's profile and a 'Solicitar trueque' (Request trade) button. The service is marked as 'Aprobado' (Approved) and 'Visible' (Visible). The page includes a 'Volver a servicios' (Return to services) link and a 'Guardar servicio' (Save service) button.

- Perfil público de usuario.

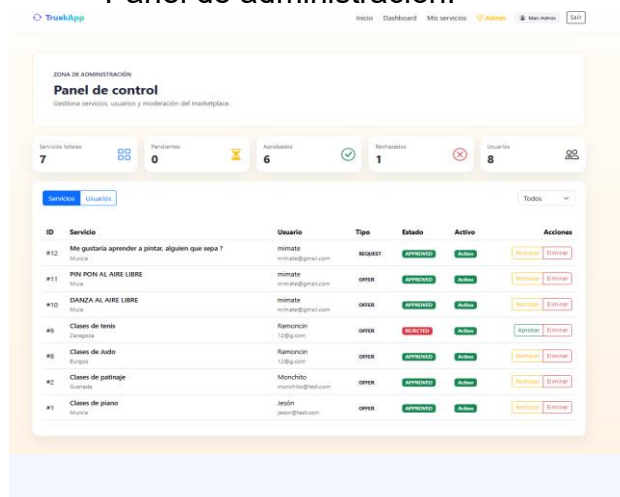
TruekApp



- Página de servicios guardados.

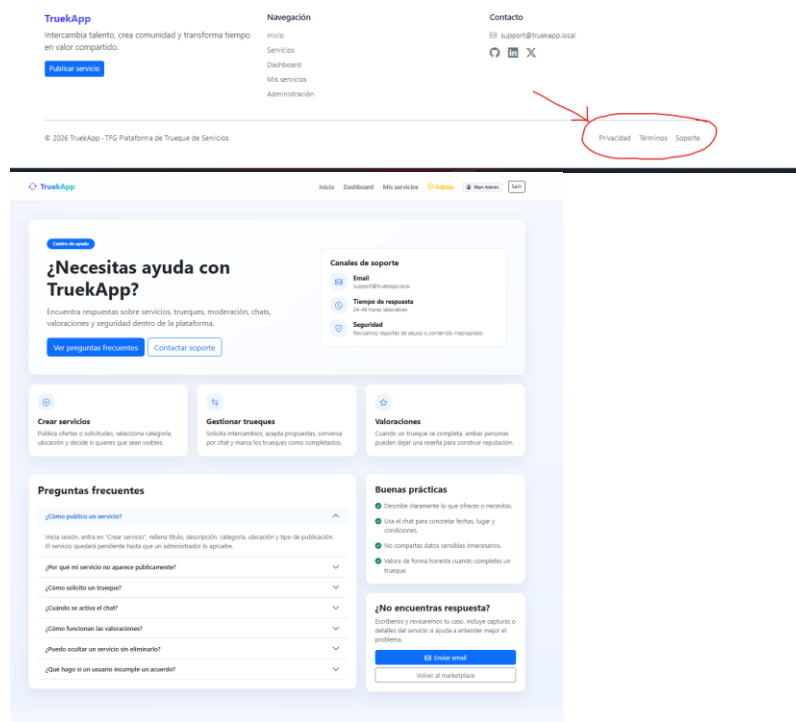


- Panel de administración.



- Páginas legales y soporte.

Para acceder a las páginas legales y soporte se debe ir al footer donde están los textos clickables con enlaces que te redirigen a las mismas.



El diseño se apoya en tarjetas, formularios, alertas, badges de estado y botones de acción para que el usuario pueda entender en todo momento qué puede hacer dentro del sistema.

4.4 Diseño de procedimientos

El sistema funciona mediante una API REST desarrollada en Laravel. El frontend realiza peticiones HTTP al backend mediante Axios (librería de JavaScript que permite gestionar solicitudes HTTP de forma sencilla), enviando y recibiendo datos en formato JSON.

Los principales procedimientos del sistema son:

- Registro de usuarios.
- Inicio y cierre de sesión.
- Consulta de usuario autenticado.
- Creación, edición y eliminación de servicios.
- Moderación de servicios por parte del administrador.
- Consulta de servicios públicos aprobados.
- Solicitud de trueques entre usuarios.
- Aceptación, rechazo, cancelación y finalización de trueques.
- Envío y recepción de mensajes dentro de un trueque aceptado.
- Creación de valoraciones tras completar un trueque.
- Guardado y eliminación de servicios favoritos.
- Consulta de perfiles públicos de usuarios a través de un servicio.
- Gestión de notificaciones.

La seguridad de estos procedimientos se basa en Laravel Sanctum, utilizando tokens Bearer para identificar al usuario autenticado. Además, las rutas protegidas requieren autenticación y algunas acciones están restringidas por rol, como ocurre con el panel de administración.

4.5 Arquitectura general del sistema

La aplicación TruekApp sigue una arquitectura cliente-servidor desacoplada, separando claramente el frontend y el backend.

El frontend ha sido desarrollado con React y Vite, encargándose de la interfaz gráfica, navegación y experiencia de usuario. Por otro lado, el backend ha sido implementado con Laravel 11 mediante una API REST encargada de la lógica de negocio, autenticación, validaciones y acceso a la base de datos MySQL.

La comunicación entre ambas capas se realiza mediante peticiones HTTP utilizando Axios.

El flujo general del sistema es el siguiente:

Usuario



Frontend React + Vite



Axios (HTTP)



API REST Laravel 11



MySQL

Cuando un usuario realiza una acción desde la interfaz, el frontend envía una petición al backend. Laravel procesa la solicitud, valida permisos y devuelve una respuesta en formato JSON que React interpreta para actualizar la interfaz.

5. Implementación

En esta fase se ha llevado a cabo el desarrollo completo del sistema TruekApp, implementando tanto el backend como el frontend, así como la integración entre ambos mediante una API REST.

El objetivo principal de esta fase ha sido transformar el diseño previamente definido en una aplicación funcional, escalable y mantenible.

5.1 Entorno de desarrollo

Para el desarrollo del proyecto se han utilizado las siguientes tecnologías y herramientas:

Backend

- **Laravel 11**: framework principal para el desarrollo del servidor.
- **PHP**: lenguaje de programación utilizado en el backend.
- **MySQL**: sistema de gestión de bases de datos.

- **Laravel Sanctum**: sistema de autenticación mediante tokens.
- **Composer**: gestor de dependencias de PHP.

Frontend

- **React + Vite**: framework para el desarrollo de la interfaz de usuario.
- **JavaScript (ES6+)**
- **Bootstrap**: librería de estilos para diseño responsive.
- **Axios**: cliente HTTP para consumir la API REST.

Herramientas adicionales

- **Visual Studio Code**: editor de código principal.
- **Git y GitHub**: control de versiones y repositorio remoto.
- **XAMPP**: entorno local para ejecución de servidor y base de datos.
- **Bruno / Postman**: pruebas de endpoints de la API.

5.2 Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor basada en una API REST:

- El **frontend** (React) se encarga de la interfaz de usuario.
- El **backend** (Laravel) gestiona la lógica de negocio y acceso a datos.
- La **base de datos** almacena la información del sistema.

La comunicación entre frontend y backend se realiza mediante peticiones HTTP en formato JSON.

Esta separación permite una mayor escalabilidad y facilita el mantenimiento del sistema a corto, medio y largo plazo.

5.3 Desarrollo del backend

El backend ha sido desarrollado utilizando Laravel, siguiendo una arquitectura basada en controladores, modelos y rutas.

Principales funcionalidades implementadas:

◇ Autenticación

- Registro de usuarios.
- Inicio de sesión.
- Cierre de sesión.
- Obtención del usuario autenticado.
- Protección de rutas mediante Laravel Sanctum.

◇ Gestión de usuarios

- Consulta de perfiles públicos.
- Verificación de usuarios.

- Bloqueo de usuarios.
- Diferenciación de roles (ADMIN y USER).

◊ **Gestión de servicios**

- Creación de servicios.
- Edición de servicios.
- Eliminación de servicios.
- Listado de servicios.
- Filtro por tipo (OFFER / REQUEST).
- Sistema de moderación:
 - Servicios pendientes.
 - Aprobación por administrador.
 - Rechazo con motivo (al usuario le llega una notificación).

◊ **Sistema de trueques**

- Creación de solicitudes de intercambio.
- Validación de servicios.
- Estados del trueque:
 - PENDING
 - ACCEPTED
 - REJECTED
 - CANCELLED
 - COMPLETED

◊ **Mensajería**

- Envío de mensajes dentro de un trueque.
- Asociación de mensajes a una solicitud de intercambio.

◊ **Valoraciones**

- Creación de valoraciones entre usuarios en formato texto.
- Puntuación del 1 al 5 en formato número.
- Restricción de una valoración por usuario y trueque.

◊ **Funcionalidades adicionales**

- Guardado de servicios favoritos.
- Sistema de notificaciones básicas.

5.4 Desarrollo del frontend

El frontend ha sido desarrollado utilizando React, estructurado en componentes reutilizables y gestionando el estado mediante contexto.

Estructura principal:

- **Componentes:** elementos reutilizables (navbar, cards, formularios).
- **Páginas:** vistas principales (home, login, dashboard, admin panel), todas con navbar y footer.
- **Contexto de autenticación:** gestión del usuario logueado.
- **Rutas protegidas:** acceso restringido según autenticación.

Principales funcionalidades del frontend:

- Registro e inicio de sesión.
- Visualización de servicios.
- Creación y edición de servicios.
- Visualización de perfil de usuario.
- Sistema de navegación mediante React Router.
- Panel de administración:
 - Gestión de usuarios.
 - Moderación de servicios.

5.5 Integración frontend-backend

La comunicación entre frontend y backend se realiza mediante Axios, configurando un cliente HTTP con:

- Base URL del backend.
- Envío de tokens en cabeceras (Authorization: Bearer).
- Uso de cookies cuando es necesario.

Las peticiones se realizan en formato JSON, permitiendo una integración fluida entre ambas capas.

Bearer es el tipo de token que se envía en la cabecera Authorization de las peticiones HTTP. En este proyecto, se utiliza junto con Laravel Sanctum para autenticar a los usuarios, permitiendo acceder a recursos protegidos sin necesidad de enviar usuario y contraseña en cada petición.

5.6 Control de errores y validaciones

Se han implementado mecanismos de control de errores tanto en frontend como en backend:

Backend:

- Validación de datos en peticiones.
- Respuestas HTTP adecuadas (200, 400, 401, 404, etc.).
- Protección de rutas mediante middleware.

Frontend:

- Validación de formularios.
- Manejo de errores en peticiones.
- Feedback visual al usuario (mensajes, alertas).

5.7 Control de versiones

El proyecto ha sido gestionado mediante Git, utilizando GitHub como repositorio remoto.

Se han realizado commits progresivos durante el desarrollo, permitiendo:

- Control de cambios.
- Recuperación de versiones anteriores.
- Organización del trabajo.

5.8 Despliegue (pendiente / futuro)

Actualmente el sistema está preparado para ser desplegado en producción, aunque en esta fase se ha trabajado en entorno local.

Como posibles opciones de despliegue se contemplan:

- **Frontend:** Vercel.
- **Backend:** servidor VPS o plataformas como Render.
- **Base de datos:** MySQL en servidor remoto.

El desarrollo del proyecto ha seguido un enfoque incremental, que me ha permitido construir funcionalidades de forma progresiva y validar cada parte del sistema antes de avanzar a las siguientes fases.

6. Pruebas

En esta fase se han realizado diferentes tipos de pruebas con el objetivo de verificar el correcto funcionamiento del sistema TruekApp, detectar posibles errores y asegurar la calidad del producto final.

Las pruebas se han centrado principalmente en validar la API REST del backend, la interacción con el frontend y el comportamiento general de la aplicación.

6.1 Tipos de pruebas realizadas

Durante el desarrollo del proyecto se han llevado a cabo los siguientes tipos de pruebas:

◊ Pruebas funcionales

Se han realizado pruebas para comprobar que cada funcionalidad del sistema cumple con los requisitos definidos.

Entre ellas:

- Registro de usuarios.
- Inicio y cierre de sesión.
- Creación, edición y eliminación de servicios.
- Visualización de servicios.
- Solicitud de trueques.
- Envío de mensajes.
- Creación de valoraciones.
- Guardado de servicios favoritos.
- Moderación de servicios por parte del administrador.

◊ Pruebas de integración

Se han realizado pruebas para verificar la correcta comunicación entre frontend y backend.

Estas pruebas han consistido en:

- Envío de peticiones HTTP desde React al backend.
- Recepción de respuestas en formato JSON.
- Validación de datos en ambos lados.
- Comprobación del flujo completo de usuario.

◊ Pruebas de autenticación

Se ha comprobado el correcto funcionamiento del sistema de autenticación mediante Laravel Sanctum:

- Acceso a rutas protegidas sin token (rechazado).
- Acceso con token válido.
- Persistencia de sesión en el frontend.
- Cierre de sesión y eliminación del token.

◊ Pruebas de validación de datos

Se han probado diferentes escenarios para asegurar que los datos introducidos por el usuario son correctos:

- Campos obligatorios vacíos.
- Emails duplicados.
- Contraseñas incorrectas.
- Datos inválidos en formularios.

6.2 Herramientas utilizadas

Para la realización de las pruebas se han utilizado las siguientes herramientas:

- **Bruno / Postman:** para probar los endpoints del backend de forma manual.

- **Navegador web:** para probar la interacción completa con la aplicación.
- **Consola del navegador (DevTools):** para detectar errores en frontend y peticiones HTTP.
- **Logs de Laravel:** para depurar errores en el backend.

6.3 Ejemplos de pruebas realizadas

A continuación se describen algunos casos de prueba representativos:

☒ **Caso de prueba: Registro de usuario**

- **Entrada:** datos válidos de usuario.
- **Resultado esperado:** usuario registrado correctamente.
- **Resultado obtenido:** registro exitoso y acceso al sistema.

☐ **Caso de prueba: Login incorrecto**

- **Entrada:** credenciales inválidas.
- **Resultado esperado:** error de autenticación.
- **Resultado obtenido:** mensaje de error y acceso denegado.

☒ **Caso de prueba: Creación de servicio**

- **Entrada:** datos completos del servicio.
- **Resultado esperado:** servicio creado en estado PENDING.
- **Resultado obtenido:** servicio almacenado correctamente y pendiente de moderación.

☒ **Caso de prueba: Moderación de servicio**

- **Entrada:** acción de aprobar o rechazar por parte del administrador.
- **Resultado esperado:** cambio de estado del servicio.
- **Resultado obtenido:** estado actualizado correctamente (APPROVED / REJECTED).

☒ **Caso de prueba: Solicitud de trueque**

- **Entrada:** selección de servicio objetivo y servicio ofrecido.
- **Resultado esperado:** creación de solicitud de intercambio.
- **Resultado obtenido:** solicitud registrada con estado PENDING.

6.4 Resultados obtenidos

Los resultados de las pruebas han sido satisfactorios, ya que el sistema ha respondido correctamente en la mayoría de los casos.

Se han detectado algunos errores durante el desarrollo, principalmente relacionados con:

- Configuración de CORS (mecanismo que permite o restringe las peticiones entre distintos orígenes).
- Problemas de autenticación.
- Validaciones en frontend.

Estos errores han sido corregidos, logrando una aplicación estable y funcional.

6.5 Modelo físico de la base de datos

El modelo físico de la base de datos ha sido implementado en MySQL utilizando migraciones de Laravel.

Cada tabla incluye:

- claves primarias (id)
- claves foráneas
- restricciones de integridad
- timestamps automáticos (created_at, updated_at)

Además, se utilizan distintos tipos de datos según las necesidades del sistema:

- VARCHAR para textos cortos
- TEXT para descripciones extensas
- ENUM para estados y roles
- BOOLEAN para estados activos/inactivos

Las relaciones entre tablas se gestionan mediante claves foráneas definidas en las migraciones del backend.

A continuación, se incluyen capturas del modelo físico generado en MySQL/phpMyAdmin a través del terminal de Visual Studio Code.

- Tabla users:

```
public function up(): void
{
    Schema::create('users', function (Blueprint $table) {
        $table->id();
        $table->string('name');
        $table->string('email')->unique();
        $table->timestamp('email_verified_at')->nullable();
        $table->string('password');
        $table->rememberToken();
        $table->timestamps();
    });

    Schema::create('password_reset_tokens', function (Blueprint $table) {
        $table->string('email')->primary();
        $table->string('token');
        $table->timestamp('created_at')->nullable();
    });

    Schema::create('sessions', function (Blueprint $table) {
        $table->string('id')->primary();
        $table->foreignId('user_id')->nullable()->index();
        $table->string('ip_address', 45)->nullable();
        $table->text('user_agent')->nullable();
        $table->longText('payload');
        $table->integer('last_activity')->index();
    });
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('users');
    Schema::dropIfExists('password_reset_tokens');
    Schema::dropIfExists('sessions');
}
};
```

- Tabla categorías:

```
public function up(): void
{
    Schema::create('categories', function (Blueprint $table) {
        $table->id();
        $table->string('name')->unique();
        $table->timestamps();
    });
}

/**
 * Reverse the migrations.
 */
public function down(): void
{
    Schema::dropIfExists('categories');
}
```

- Tabla servicios:

```
public function up(): void
{
    Schema::create('services', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->cascadeOnDelete();
        $table->foreignId('category_id')->nullable()->constrained()->nullOnDelete();

        $table->enum('type', ['OFFER', 'REQUEST']);
        $table->string('title');
        $table->text('description')->nullable();
        $table->string('location')->nullable();

        $table->boolean('is_active')->default(true);
        $table->timestamps();

        $table->index(['type', 'category_id', 'is_active']);
    });
}
```

- Tabla trueques:

```
public function up(): void
{
    Schema::create('trade_requests', function (Blueprint $table) {
        $table->id();

        $table->foreignId('requester_id')->constrained('users')->cascadeOnDelete();
        $table->foreignId('target_service_id')->constrained('services')->cascadeOnDelete();
        $table->foreignId('offer_service_id')->nullable()->constrained('services')->nullOnDelete();

        $table->enum('status', ['PENDING', 'ACCEPTED', 'REJECTED', 'CANCELLED', 'COMPLETED'])->default('PENDING');
        $table->text('message')->nullable();

        $table->timestamps();

        $table->unique(['requester_id', 'target_service_id']);
    });
}
```

- Tabla reviews:


```
Schema::create('reviews', function (Blueprint $table) {
    $table->id();

    $table->foreignId('from_user_id')->constrained('users')->cascadeOnDelete();
    $table->foreignId('to_user_id')->constrained('users')->cascadeOnDelete();
    $table->foreignId('trade_request_id')->constrained('trade_requests')->cascadeOnDelete();

    $table->unsignedTinyInteger('rating'); // 1-5
    $table->text('comment')->nullable();

    $table->timestamps();

    $table->unique(['from_user_id', 'trade_request_id']);
});
```

- Tabla roles y autenticaciones:

```
Schema::table('services', function (Blueprint $table) {
    $table->enum('moderation_status', ['PENDING', 'APPROVED', 'REJECTED'])->default('PENDING');
    $table->foreignId('reviewed_by')->nullable()->constrained('users')->nullOnDelete();
    $table->timestamp('reviewed_at')->nullable();
    $table->text('rejection_reason')->nullable();
});
```

- Tabla estado de los servicios:

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->enum('role', ['ADMIN', 'USER'])->default('USER');
        $table->enum('status', ['PENDING', 'APPROVED', 'REJECTED'])->default('PENDING');
    });
}
```

- Tabla de los tokens de usuario:

```
public function up(): void
{
    Schema::create('personal_access_tokens', function (Blueprint $table) {
        $table->id();
        $table->morphs('tokenable');
        $table->text('name');
        $table->string('token', 64)->unique();
        $table->text('abilities')->nullable();
        $table->timestamp('last_used_at')->nullable();
        $table->timestamp('expires_at')->nullable()->index();
        $table->timestamps();
    });
}
```

- Tabla notificaciones:

```
public function up(): void
{
    Schema::create('notifications', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->cascadeOnDelete();
        $table->string('type')->default('INFO');
        $table->string('title');
        $table->text('message');
        $table->timestamp('read_at')->nullable();
        $table->timestamps();
    });
}
```

- Tabla de mensajes entre usuarios:

```
public function up(): void
{
    Schema::create('trade_messages', function (Blueprint $table) {
        $table->id();
        $table->foreignId('trade_request_id')->constrained()->cascadeOnDelete();
        $table->foreignId('sender_id')->constrained('users')->cascadeOnDelete();
        $table->text('message');
        $table->timestamps();
    });
}

public function down(): void
```

- Tabla de la foto de perfil del usuario

```
public function up(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->string('avatar_path')->nullable()->after('status');
    });
}

public function down(): void
{
    Schema::table('users', function (Blueprint $table) {
        $table->dropColumn('avatar_path');
    });
}
```

- Tabla de servicios guardados:

```
public function up(): void
{
    Schema::create('saved_services', function (Blueprint $table) {
        $table->id();
        $table->foreignId('user_id')->constrained()->cascadeOnDelete();
        $table->foreignId('service_id')->constrained()->cascadeOnDelete();
        $table->timestamps();

        $table->unique(['user_id', 'service_id']);
    });
}
```

- Tabla de estados de verificación/bloqueo de usuarios por admin

```
return new class extends Migration
{
    public function up(): void
    {
        DB::statement("ALTER TABLE users MODIFY status ENUM('PENDING', 'APPROVED', 'REJECTED', 'VERIFIED', 'BLOCKED') DEFAULT 'PENDING'");
    }

    public function down(): void
    {
        DB::statement("ALTER TABLE users MODIFY status ENUM('PENDING', 'APPROVED', 'REJECTED') DEFAULT 'PENDING'");
    }
};
```

6.6 Ley de protección de datos

TruekApp almacena información básica de los usuarios necesaria para el funcionamiento de la plataforma, como nombre, correo electrónico y credenciales de acceso.

Con el objetivo de garantizar la seguridad y privacidad de los datos, se han aplicado diferentes medidas de protección:

- Contraseñas cifradas mediante hashing.
- Acceso protegido mediante autenticación con tokens.
- Restricción de rutas privadas mediante middleware.
- Validación de permisos según el rol del usuario.
- Separación entre frontend y backend para mejorar el control de acceso.
- Validación de datos enviados desde cliente y servidor.

El sistema sigue principios alineados con el Reglamento General de Protección de Datos (RGPD), evitando almacenar información innecesaria y limitando el acceso a los datos únicamente a usuarios autorizados.

Además, el usuario puede modificar o eliminar parte de su información desde la propia plataforma.

6.7 Conclusión

Las pruebas realizadas han permitido validar el correcto funcionamiento del sistema TruekApp, asegurando que las funcionalidades principales operan según lo esperado. Además, han contribuido a mejorar la estabilidad del sistema y a detectar posibles fallos antes de su puesta en producción.

Como mejora futura, se propone la implementación de pruebas automatizadas mediante herramientas como PHPUnit o Jest (frameworks utilizados para realizar pruebas automatizadas del código), que permitan validar el sistema de forma más eficiente y continua.

7. Documentación

La documentación del proyecto TruekApp tiene como objetivo facilitar la comprensión, uso y mantenimiento del sistema, tanto para usuarios finales como para desarrolladores.

Se ha dividido en documentación técnica y documentación de usuario, cubriendo todos los aspectos necesarios para el correcto funcionamiento y evolución de la aplicación.

7.1 Documentación técnica

La documentación técnica describe la estructura interna del sistema, las tecnologías utilizadas y el funcionamiento de sus distintos componentes. Incluye los siguientes elementos:

◊ Arquitectura del sistema

Se describe la arquitectura cliente-servidor utilizada en el proyecto:

- Frontend desarrollado en React.
- Backend desarrollado en Laravel mediante API REST.
- Base de datos MySQL.

Se detalla la separación de responsabilidades entre cada capa, facilitando la escalabilidad y el mantenimiento del sistema.

◊ Estructura del backend

El backend sigue el patrón MVC (Modelo-Vista-Controlador), organizado en:

- **Modelos:** representan las entidades de la base de datos.
- **Controladores:** gestionan la lógica de negocio.
- **Rutas:** definen los endpoints de la API.
- **Middleware:** controlan el acceso y la seguridad.

También se documenta el uso de Laravel Sanctum para la autenticación mediante tokens.

◊ Estructura del frontend

El frontend está organizado en:

- **Componentes reutilizables** (navbar, formularios, tarjetas).
- **Páginas principales** (home, login, dashboard, admin panel).
- **Contexto de autenticación** para gestión de sesión.
- **Sistema de rutas** mediante React Router.

Esto permite una organización modular del código y facilita su ampliación.

◊ API REST

Se documentan los principales endpoints del sistema, incluyendo:

- Autenticación (/api/register, /api/login, /api/logout).
- Servicios (/api/services).
- Usuarios (/api/users).
- Trueques (/api/trade-requests).
- Valoraciones (/api/reviews).

Para cada endpoint se especifica:

- Método HTTP.
- Parámetros requeridos.
- Respuesta esperada.

◇ Base de datos

Se incluye el modelo entidad-relación del sistema, así como la descripción de las tablas principales:

- users
- services
- categories
- trade_requests
- reviews
- notifications
- trade_messages
- saved_services

También se documentan las relaciones entre tablas y las restricciones aplicadas.

7.2 Documentación de usuario

La documentación de usuario describe cómo interactuar con el sistema desde el punto de vista del usuario final.

◇ Registro e inicio de sesión

El usuario puede crear una cuenta proporcionando sus datos personales. Una vez registrado, puede iniciar sesión para acceder a las funcionalidades del sistema.

◇ Uso de la plataforma

El usuario puede:

- Explorar servicios disponibles.
- Crear servicios propios.
- Editar o eliminar sus servicios.
- Solicitar intercambios con otros usuarios.
- Comunicarse mediante mensajes dentro de un trueque.
- Valorar a otros usuarios tras completar un intercambio.
- Guardar servicios como favoritos.

◇ Panel de administración

Los usuarios con rol ADMIN pueden acceder a un panel de administración desde el cual pueden:

- Gestionar usuarios.
- Verificar usuarios.
- Bloquear usuarios.
- Moderar servicios (aprobar o rechazar).

7.3 Mantenimiento del sistema

El sistema se ha desarrollado para facilitar su mantenimiento y evolución futura.

Se han aplicado buenas prácticas como:

- Separación de responsabilidades.
- Uso de control de versiones (Git).
- Organización modular del código.
- Uso de frameworks consolidados.

◇ Posibles mejoras futuras

Se identifican varias líneas de mejora:

- Implementación de pruebas automatizadas.
- Mejora del sistema de notificaciones.
- Desarrollo de un panel de estadísticas.
- Implementación de funcionalidades adicionales en el panel de administración.
- Optimización del rendimiento.

7.5 Ley de protección de datos

TruekApp trata información personal de los usuarios, como nombre, correo electrónico y datos relacionados con los servicios publicados dentro de la plataforma. Durante el desarrollo del proyecto se han aplicado medidas básicas de seguridad orientadas a proteger la información almacenada en el sistema.

Las contraseñas de los usuarios se almacenan cifradas mediante los mecanismos proporcionados por Laravel, evitando guardar información sensible en texto plano. Además, el acceso a determinadas funcionalidades está restringido mediante autenticación y control de roles, utilizando Laravel Sanctum para gestionar sesiones seguras mediante tokens.

En un entorno real de producción, sería necesario adaptar el sistema al Reglamento General de Protección de Datos (RGPD), incorporando políticas de privacidad, consentimiento explícito, gestión de eliminación de datos y mecanismos adicionales de protección.

7.4 Conclusión

La documentación generada permite comprender el funcionamiento del sistema TruekApp, facilitando tanto su uso como su mantenimiento. Además, proporciona una base sólida para futuras ampliaciones del proyecto.

8. Mantenimiento

El mantenimiento del sistema TruekApp tiene como objetivo garantizar su correcto funcionamiento a lo largo del tiempo, así como facilitar su evolución, adaptación y mejora continua.

Se han definido una serie de estrategias que permiten asegurar la estabilidad del sistema, la protección de los datos y la posibilidad de escalar la aplicación en el futuro.

8.1 Planificación de copias de seguridad

Para garantizar la seguridad de la información almacenada en la base de datos, es necesario establecer un sistema de copias de seguridad.

◇ Tipos de copias

- **Copias completas:** respaldo total de la base de datos en un momento determinado.
- **Copias incrementales:** almacenamiento únicamente de los cambios realizados desde la última copia.

◇ Frecuencia

Se propone la siguiente planificación:

- Copia diaria automática de la base de datos.
- Copia semanal completa almacenada externamente.
- Copias adicionales antes de realizar cambios importantes en el sistema.

◇ Almacenamiento

Las copias de seguridad deben almacenarse en:

- Servidores externos o servicios en la nube.
- Ubicaciones distintas al servidor principal.

Esto permite recuperar la información en caso de fallo del sistema.

8.2 Escalabilidad del sistema

El sistema ha sido diseñado para permitir su crecimiento y adaptación a un mayor número de usuarios.

◇ **Características que facilitan la escalabilidad**

- Arquitectura cliente-servidor desacoplada.
- Uso de una API REST reutilizable.
- Separación entre frontend y backend.
- Uso de base de datos relacional estructurada.

◇ **Posibles mejoras de escalabilidad**

- Migración a servidores en la nube.
- Uso de balanceadores de carga.
- Optimización de consultas a base de datos.
- Implementación de caché para mejorar el rendimiento.

8.3 Medidas de seguridad

Para proteger el sistema y los datos de los usuarios, se han aplicado diversas medidas de seguridad:

◇ **Autenticación y control de acceso**

- Uso de Laravel Sanctum para autenticación mediante tokens.
- Protección de rutas mediante middleware.
- Control de roles (USER y ADMIN).

◇ **Protección de datos**

- Almacenamiento seguro de contraseñas mediante cifrado.
- Validación de datos en backend y frontend.
- Uso de claves foráneas para mantener la integridad de la base de datos.

◇ **Seguridad en la aplicación**

- Control de acceso a funcionalidades según rol.
- Validación de formularios para evitar datos incorrectos.
- Manejo de errores para evitar exposición de información sensible.

8.4 Mantenimiento evolutivo

El sistema está preparado para evolucionar con nuevas funcionalidades en el futuro. Entre las posibles mejoras se incluyen:

- Sistema de notificaciones en tiempo real.

- Aplicación móvil.
- Mejora del sistema de recomendaciones.
- Panel de estadísticas para administradores.
- Ampliación de funcionalidades del panel de administración.

8.5 Conclusión

El mantenimiento del sistema TruekApp garantiza su estabilidad, seguridad y capacidad de crecimiento.

Gracias a la estructura modular del proyecto y al uso de tecnologías actuales, la aplicación puede evolucionar fácilmente para adaptarse a nuevas necesidades.

9. Mejoras futuras

Aunque el sistema TruekApp se encuentra funcionalmente completo en su versión actual, existen diversas líneas de mejora que permitirían ampliar sus capacidades, optimizar su rendimiento y mejorar la experiencia del usuario.

Estas mejoras se plantean como posibles evoluciones del proyecto en futuras versiones.

9.1 Mejoras funcionales

Se identifican varias funcionalidades que podrían incorporarse para enriquecer el sistema:

- Implementación de **notificaciones en tiempo real**, permitiendo a los usuarios recibir avisos instantáneos sobre solicitudes, mensajes o valoraciones.
- Desarrollo de un **sistema de búsqueda avanzada**, con filtros más precisos y ordenación dinámica.
- Incorporación de un sistema de reputación más completo, incluyendo estadísticas de usuario.
- Mejora del sistema de recomendaciones mediante algoritmos más avanzados.
- Posibilidad de añadir imágenes o archivos a los servicios.

9.2 Mejora del panel de administración

El panel de administración actual se centra en la gestión de usuarios y la moderación de servicios, pero puede ampliarse con nuevas funcionalidades:

- Gestión completa de categorías (crear, editar y eliminar).
- Supervisión de trueques y posibilidad de intervención en casos conflictivos.
- Moderación de mensajes y valoraciones.
- Panel de estadísticas del sistema (usuarios, servicios, actividad).
- Exportación de datos en diferentes formatos.

9.3 Mejora de la experiencia de usuario

Para mejorar la usabilidad de la aplicación se proponen:

- Optimización del diseño visual y experiencia de navegación.
- Mejora de los tiempos de carga.
- Implementación de animaciones e interacciones más dinámicas.
- Adaptación completa a dispositivos móviles (mobile-first).
- Mejora del feedback visual en acciones del usuario.

9.4 Escalabilidad y rendimiento

A nivel técnico, se pueden implementar mejoras para soportar un mayor volumen de usuarios:

- Despliegue en infraestructuras cloud.
- Uso de sistemas de caché para optimizar consultas.
- Implementación de balanceadores de carga.
- Optimización de consultas a la base de datos.

9.5 Desarrollo de nuevas plataformas

Como evolución del proyecto, se plantea:

- Desarrollo de una aplicación móvil nativa.
- Creación de una versión progresiva (PWA).
- Integración con servicios externos o APIs.

9.6 Pruebas automatizadas

Actualmente las pruebas se han realizado manualmente, pero se propone:

- Implementación de pruebas automatizadas en backend (PHPUnit).
- Pruebas en frontend (Jest o similares).

9.7 Conclusión

Las mejoras propuestas permiten ampliar significativamente el potencial del sistema TruekApp, aportando nuevas funcionalidades, mejorando la experiencia del usuario y facilitando la evolución del proyecto a largo plazo.

Estas líneas de desarrollo convierten la aplicación en una plataforma con capacidad de crecimiento y adaptación a futuros requisitos.

10. Conclusiones

El desarrollo del proyecto TruekApp ha permitido aplicar de forma práctica los conocimientos adquiridos durante el ciclo formativo de Desarrollo de Aplicaciones Web, abarcando tanto el desarrollo backend como frontend, así como el diseño de bases de datos y la integración de sistemas.

El objetivo principal del proyecto, consistente en crear una plataforma de intercambio de servicios sin uso de dinero, se ha cumplido satisfactoriamente, dando lugar a una aplicación funcional que permite a los usuarios publicar servicios, solicitar intercambios, comunicarse y valorar a otros usuarios tras completar un trueque.

10.1 Valoración del proyecto

El resultado final es una aplicación completa que incluye:

- Sistema de autenticación de usuarios.
- Gestión de servicios (creación, edición, eliminación).
- Sistema de moderación de servicios.
- Solicitudes de intercambio entre usuarios.
- Sistema de mensajería asociado a trueques.
- Sistema de valoraciones.
- Funcionalidad de guardado de servicios.
- Panel de administración para gestión de usuarios y servicios.

El sistema ha sido desarrollado siguiendo buenas prácticas, utilizando tecnologías actuales como Laravel, React y MySQL, lo que ha permitido obtener una aplicación estructurada, mantenible y escalable.

10.2 Dificultades encontradas

Durante el desarrollo del proyecto se han presentado diversas dificultades, entre las que destacan:

- Configuración de la autenticación entre frontend y backend mediante Laravel Sanctum.
- Problemas de CORS en la comunicación entre cliente y servidor.
- Gestión del estado en el frontend.
- Integración entre las distintas funcionalidades del sistema.
- Diseño de un sistema de matching y recomendaciones eficientes.

Estas dificultades han sido superadas mediante investigación, pruebas y depuración del código.

10.3 Conocimientos adquiridos

El desarrollo del proyecto ha permitido consolidar y ampliar conocimientos en diferentes áreas:

- Desarrollo de APIs REST con Laravel.
- Uso de React para interfaces dinámicas.
- Gestión de bases de datos relacionales.
- Autenticación y seguridad en aplicaciones web.
- Integración entre frontend y backend.
- Control de versiones con Git.

10.4 Valoración personal

Este proyecto se ha supuesto una experiencia interesante y de valor, a nivel técnico y personal.

Ha permitido desarrollar habilidades de resolución de problemas, organización del trabajo y autonomía, así como mejorar la capacidad para afrontar proyectos complejos desde su fase inicial hasta su finalización.

10.5 Anexos

Repositorio del proyecto

Repositorio GitHub del proyecto:

- Backend Laravel 11
- Frontend React + Vite
- Documentación técnica
- Estructura completa del proyecto

En el zip se adjunta el proyecto entero (back-end, front-end, base de datos exportada y un readme con un resumen del programa)

10.6 Conclusión final

TruekApp representa una aplicación completa y funcional que cumple con los objetivos planteados inicialmente, ofreciendo una solución real al intercambio de servicios entre usuarios sin necesidad de dinero.

El proyecto ha sido desarrollado con una base sólida que permite su evolución futura, lo que abre la puerta a nuevas funcionalidades y mejoras en versiones posteriores.

11. Bibliografía

En este apartado se recogen las principales fuentes de información utilizadas durante el desarrollo del proyecto TruekApp. Estas fuentes han servido de apoyo para la implementación técnica, la resolución de problemas y la comprensión de las tecnologías empleadas.

11.1 Documentación oficial

- | | |
|---|----------------|
| • Laravel | Documentation. |
| https://laravel.com/docs | |
| • React | Documentation. |
| https://react.dev | |
| • Vite | Documentation. |
| https://vitejs.dev | |
| • Bootstrap | Documentation. |
| https://getbootstrap.com | |

- Axios Documentation.
<https://axios-http.com>
- MySQL Documentation.
<https://dev.mysql.com/doc>

11.2 Recursos técnicos y aprendizaje

- Stack Overflow.
<https://stackoverflow.com>
- MDN Web Docs.
<https://developer.mozilla.org>
- W3Schools.
<https://www.w3schools.com>
- OpenAI ChatGPT.
<https://chat.openai.com>
(Herramienta utilizada como apoyo para resolver dudas técnicas, generación de ideas, optimización y ayuda de diseño).
- YouTube
https://www.youtube.com/watch?v=DTi_fqbH_aM
(Herramienta utilizada como apoyo para entender bien el funcionamiento y utilidad de Sanctum).
- https://www.youtube.com/watch?v=7iobxzd_2wY&list=PLUofhDIg_38q4D0xN_Wp7FEHOTcZhjWJ29
(Herramienta utilizada como apoyo para entender bien el funcionamiento, implementación y utilidad de React).

11.3 Herramientas utilizadas

- GitHub.
<https://github.com>
- Visual Studio Code.
<https://code.visualstudio.com>
- XAMPP.
<https://www.apachefriends.org>
- Bruno (API Client).
<https://www.usebruno.com>

11.4 Recursos adicionales

- Tutoriales y documentación online relacionados con desarrollo web, APIs REST y frameworks modernos.
- Material didáctico proporcionado durante el ciclo formativo de Desarrollo de Aplicaciones Web.

